

Project: Modular Utility Library with Error Handling

Project Scenario

Tum ek startup ke liye reusable utility library bana rahe ho.

Library mein hoga: validators, formatters, api-client, aur error classes —

sab properly modular (separate files), proper error handling ke saath.

Final output: ek barrel index.js se sab kuch import — clean public API.

Ye exactly wahi pattern hai jo lodash, axios, aur React ka source code follow karta hai.

Project Structure

```
utils-library/  
  ■■■■ src/  
  ■ ■■■■ errors/  
  ■ ■ ■■■■ AppError.js  
  ■ ■ ■■■■ ValidationError.js  
  ■ ■ ■■■■ index.js  
  ■ ■■■■ validators/  
  ■ ■ ■■■■ string.validators.js  
  ■ ■ ■■■■ number.validators.js  
  ■ ■ ■■■■ index.js  
  ■ ■■■■ formatters/  
  ■ ■ ■■■■ date.formatter.js  
  ■ ■ ■■■■ currency.formatter.js  
  ■ ■ ■■■■ index.js  
  ■ ■■■■ api/  
  ■ ■ ■■■■ client.js  
  ■ ■ ■■■■ interceptors.js  
  ■ ■■■■ index.js <-- main barrel  
  ■■■■ test/  
  ■ ■■■■ validators.test.js  
  ■■■■ package.json (type: module)
```

Part 1 — Error Module (src/errors/)

Error Module

```
// src/errors/AppError.js  
export class AppError extends Error {  
  constructor(message, code, statusCode = 500, details = {}) {
```

```

super(message);
this.name = this.constructor.name;
this.code = code;
this.statusCode = statusCode;
this.details = details;
this.timestamp = new Date().toISOString();
this.isOperational = true;
if (Error.captureStackTrace) {
  Error.captureStackTrace(this, this.constructor);
}
}
}
toJSON() {
  return {
    name: this.name, code: this.code,
    message: this.message, statusCode: this.statusCode,
    details: this.details, timestamp: this.timestamp
  };
}
}
// src/errors/ValidationError.js
import { AppError } from './AppError.js';
export class ValidationError extends AppError {
  constructor(errors) { // errors = array of {field, message}
    super('Validation failed', 'VALIDATION_ERROR', 400, { errors });
    this.errors = errors;
  }
  static single(field, message) {
    return new ValidationError([{ field, message }]);
  }
}
export class NotFoundError extends AppError {
  constructor(resource, identifier) {
    super(`${resource} '${identifier}' not found`, 'NOT_FOUND', 404, { resource, identifier });
  }
}
export class AuthError extends AppError {
  constructor(msg = 'Authentication required') {
    super(msg, 'AUTH_ERROR', 401);
  }
}
// src/errors/index.js – barrel
export { AppError } from './AppError.js';
export { ValidationError, NotFoundError, AuthError } from './ValidationError.js';

```

Part 2 — Validators Module (src/validators/)

Validators Module

```
// src/validators/string.validators.js
```

```

import { ValidationError } from '../errors/index.js';
export const isEmail = (v) =>
/^([a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,})$/.test(v);
export const isPhone = (v) => /^[6-9]\d{9}$/.test(String(v));
export const isURL = (v) => {
  try { new URL(v); return true; }
  catch { return false; }
};
export const isStrongPassword = (v) => ({
  valid: v.length >= 8 && /[A-Z]/.test(v) && /[0-9]/.test(v) && /[a-zA-Z0-9]/.test(v),
  checks: {
    length: v.length >= 8,
    uppercase: /[A-Z]/.test(v),
    number: /[0-9]/.test(v),
    special: /^[a-zA-Z0-9]/.test(v),
  }
});
// Schema validator – validate entire object at once
export function validateSchema(data, schema) {
  const errors = [];
  for (const [field, rules] of Object.entries(schema)) {
    const value = data[field];
    if (rules.required && (value === undefined || value === null || value === ''))
      errors.push({ field, message: `${field} is required` });
    if (value && rules.type && typeof value !== rules.type)
      errors.push({ field, message: `${field} must be ${rules.type}` });
    if (value && rules.minLength && value.length < rules.minLength)
      errors.push({ field, message: `${field} min length: ${rules.minLength}` });
    if (value && rules.maxLength && value.length > rules.maxLength)
      errors.push({ field, message: `${field} max length: ${rules.maxLength}` });
    if (value && rules.pattern && !rules.pattern.test(value))
      errors.push({ field, message: rules.patternMsg || `${field} format invalid` });
    if (value && rules.custom) {
      const customErr = rules.custom(value);
      if (customErr) errors.push({ field, message: customErr });
    }
  }
  if (errors.length > 0) throw new ValidationError(errors);
  return true;
}

```

Part 3 — Formatters Module (src/formatters/)

Formatters Module

```

// src/formatters/currency.formatter.js
export const formatINR = (amount, options = {}) => {
  const { showSymbol = true, decimals = 2 } = options;
  return new Intl.NumberFormat('en-IN', {

```

```

style: showSymbol ? 'currency' : 'decimal',
currency: 'INR',
minimumFractionDigits: decimals,
}).format(amount);
};

export const parseINR = (str) => parseFloat(str.replace(/^[^0-9.]/g, ''));

// src/formatters/date.formatter.js
export const formatDate = (date, locale = 'en-IN') => {
  try {
    return new Intl.DateTimeFormat(locale, {
      day: '2-digit', month: 'short', year: 'numeric'
    }).format(new Date(date));
  } catch {
    throw new Error(`Invalid date: ${date}`);
  }
};

export const timeAgo = (date) => {
  const diff = Date.now() - new Date(date).getTime();
  const minutes = Math.floor(diff / 60000);
  if (minutes < 1) return 'just now';
  if (minutes < 60) return `${minutes}m ago`;
  const hours = Math.floor(minutes / 60);
  if (hours < 24) return `${hours}h ago`;
  const days = Math.floor(hours / 24);
  return `${days}d ago`;
};

// src/formatters/index.js
export * from './currency.formatter.js';
export * from './date.formatter.js';

```

Part 4 — API Client with Error Handling (src/api/)

API Client — Complete with Error Handling

```

// src/api/client.js
import { NetworkError, NotFoundError, AuthError, AppError } from '../errors/index.js';

class APIClient {
  #baseUrl; #headers; #timeout;

  constructor(baseUrl, options = {}) {
    this.#baseUrl = baseUrl.replace(/\/$/, '');
    this.#headers = { 'Content-Type': 'application/json', ...options.headers };
    this.#timeout = options.timeout || 10000;
  }

  setToken(token) {
    this.#headers['Authorization'] = `Bearer ${token}`;
    return this;
  }

  async #request(method, endpoint, body = null) {
    const url = `${this.#baseUrl}${endpoint}`;

```



```

email: { required: true, pattern: /^[^@]+@[^@]+\.[^@]+$/, patternMsg: 'Invalid email' },
password: { required: true, custom: v => isStrongPassword(v).valid ? null : 'Password too weak' },
age: { required: true, type: 'number' },
};

try {
  validateSchema({ name: 'Rahul', email: 'rahul@test.com', password: 'Weak1!', age: 20 },
    userSchema);
  console.log('Validation passed!');
} catch (e) {
  if (e instanceof ValidationError) {
    console.log('Errors:', e.errors);
  }
}

// Use API client
const api = new APIClient('https://jsonplaceholder.typicode.com');

async function main() {
  try {
    const user = await api.get('/users/1');
    console.log(user.name);
    console.log(formatINR(50000)); // '₹50,000.00'
    console.log(formatDate(new Date())); // '11 Jun 2026'
  } catch (e) {
    if (e instanceof AuthError) console.log('Login required');
    if (e instanceof NotFoundError) console.log('Not found:', e.resource);
    else console.log('Error:', e.message);
  }
}

main();

```

Bonus Challenges

1. `retry(fn, maxAttempts, delay)` — failed API calls automatically retry karo.
2. `RateLimiter` class banao — X requests per minute allow karo, baaki queue karo.
3. `cache()` HOF banao — API results TTL ke saath cache karo.
4. Dynamic import use karo — heavy formatters sirf jab needed ho tab load karo.
5. `test/` folder mein manual test cases likho — har validator ke liye.